16. Testing Shell Scripts and Safe Automation

16. Testing Shell Scripts and Safe Automation

Phase: Phase 5 — Shell scripting
Estimated time: 1–2 days
Slug: `16-bash-testing`

Learning objectives

- Structure scripts for testability
- Write simple assert helpers
- Use dry-run flags
- Avoid destructive defaults

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

Automation that runs while you sleep must be careful. A script that does `rm -rf "$dir"/*` with an empty `$dir` becomes `rm -rf /*`. Real incidents have happened exactly this way. Safe shell means refusing to continue when inputs are missing, failing loudly instead of silently, and testing each condition explicitly.

This lesson covers the test command `[ ]` and `[[ ]]`, strict mode with `set -euo pipefail`, guard functions that stop a script before it does harm, and basic ways to test scripts: dry-run flags, `shellcheck`, and tiny assertion helpers.

Core concepts

Concept	Syntax	Meaning
File test	`[ -d "$d" ]`	True if `$d` is a directory
String test	`[ -n "$s" ]` / `[ -z "$s" ]`	Non-empty / empty
Numeric test	`[ "$a" -eq 3 ]`	Integer compare
Bash test	`[[ $s == abc* ]]`	Pattern match, no word splitting
Exit on error	`set -e`	Stop when a command fails
Unset is error	`set -u`	Using an undefined variable fails
Pipe failures	`set -o pipefail`	Pipeline fails if any stage fails
Trap	`trap 'cleanup' EXIT`	Run code on exit
Linter	`shellcheck script.sh`	Static analysis of bad patterns

Layers of defence:

```
user input / env
      |
      v
+------------------+   empty? not a dir? -> die (return 1)
| guard function   |
+------------------+
      | valid
      v
+------------------+   set -euo pipefail
| strict mode      |   stops on first surprise
+------------------+
      |
      v
+------------------+   --dry-run prints, does not run
| dry-run switch   |
+------------------+
      |
      v
real action (rm, mv, rsync)
```

Worked examples

A guard that refuses empty input and echoes the value otherwise:

```
safe_dir_or_die() {
  local d=${1:-}
  if [ -z "$d" ]; then
    echo "error: directory required" >&2
    return 1
  fi
  echo "$d"
}

safe_dir_or_die /tmp
echo "status=$?"
safe_dir_or_die
echo "status=$?"
```

Sample output:

```
/tmp
status=0
error: directory required
status=1
```

Using it before a dangerous action:

```
target=$(safe_dir_or_die "$1") || exit 1
rm -rf -- "${target:?}"/old-*
```

The `${var:?msg}` form aborts when the variable is unset or empty:

```
unset dir
bash -c 'rm -rf "${dir:?dir is empty}"/x'; echo "rc=$?"
```

Sample output:

```
bash: line 1: dir: dir is empty
rc=1
```

Strict mode catching a typo:

```
bash -c 'set -u; echo "$undefined_name"'; echo "rc=$?"
```

Sample output:

```
bash: line 1: undefined_name: unbound variable
rc=1
```

Cleanup with trap and a dry-run flag:

```
tmp=$(mktemp -d)
trap 'rm -rf -- "$tmp"' EXIT
DRY=${DRY:-0}
run() { if [ "$DRY" = 1 ]; then echo "+ $*"; else "$@"; fi; }
DRY=1 run mv /var/log/app.log /var/log/app.log.1
```

Sample output:

```
+ mv /var/log/app.log /var/log/app.log.1
```

Mini-lab

1. Write `safe_dir_or_die` and test it with an empty string, no argument, and `/tmp`.
2. Add a second check that the path exists with `[ -d "$d" ]`; make it return 1 for `/does/not/exist`.
3. Create `cleanup.sh` using `set -euo pipefail` and a `mktemp -d` workspace plus `trap ... EXIT`.
4. Add a `DRY` switch so `DRY=1 ./cleanup.sh` only prints actions.
5. Trigger a failure on purpose (`false` mid-script) and watch `set -e` stop it; then add `|| true` where failure is acceptable.
6. Run `shellcheck cleanup.sh` if installed (`apt install shellcheck`) and fix every warning.
7. Write a tiny `assert_eq` function and use it to test `safe_dir_or_die`.

Decision guide / failure modes

Symptom	Likely cause	Fix
Script continues after error	No `set -e`	Add strict mode, or check `$?`
`set -e` does not stop in `if`	Commands in conditions are exempt	Test explicitly
`grep` kills script with `set -e`	No match returns 1	`grep ... || true`
`[ $x = a ]` breaks on empty	Unquoted variable	`[ "$x" = a ]`
Deleted wrong files	Empty variable in path	`${var:?}` and guard functions
Temp files left behind	No cleanup on failure	`trap ... EXIT`

Strict mode is a safety net, not a design. Prefer explicit guards for the cases you know, keep dry-run available for anything destructive, and always use `--` before user-supplied paths so a name like `-rf` is not read as an option.

Common mistakes

- Empty-variable rm -rf disasters
- Untested sed -i in prod
- Scripts that only work interactively

Exercises

Work in `exercises/16-bash-testing/`. Read that folder's `README.md` fully before coding.

```
cd exercises/16-bash-testing
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Implement safe_dir_or_die in exercise.sh. It takes one argument, the directory name. If the argument is missing or an empty string, the function must return 1 (you may write an error to stderr but print nothing to stdout). Otherwise it must print the argument unchanged on stdout and return 0. Do not check whether the directory exists; this exercise only guards against empty input. Quote the variable everywhere. Run ./test.sh, then try the function in a script that uses set -u to confirm it never touches an unset variable.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/16-bash-testing/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. What is a dry-run?
2. Why guard empty paths?
3. How assert equality in bash?

Further reading

- https://www.shellcheck.net/